

dogmatch_ai

Relazione Tecnica Discorsiva

Come funziona il servizio AI - con snippet di codice reale

Giugno 2026

Questo documento spiega in modo discorsivo come funziona dogmatch_ai: come si avvia, come autentica le richieste, come rileva l'intenzione dell'utente, come costruisce il prompt per GPT-4o, come esegue il loop di tool-calling e come funzionano i tool piu' importanti. Ogni sezione e' accompagnata da snippet del codice reale.

1. Come si avvia il servizio: main.py e il lifespan

dogmatch_ai è un microservizio Python costruito su FastAPI. Il suo punto di ingresso è main.py, che configura l'applicazione, i middleware e gli endpoint. Il meccanismo più importante è il lifespan: una funzione asincrona che FastAPI esegue una volta all'avvio (startup) e una volta allo spegnimento (shutdown). È qui che vengono inizializzate tutte le risorse condivise.

1.1 Il lifespan: startup e shutdown

Al momento dello startup, il lifespan esegue in sequenza una serie di operazioni asincrone con await: crea il pool di connessioni MySQL, garantisce che tutte le tabelle necessarie esistano (creandole se mancanti), carica la knowledge base dal file JSON nel database e inserisce le 6 aree cani di seed. Infine avvia il background task per le notifiche proattive. Allo shutdown annulla questo task e chiude il pool.

main.py - lifespan FastAPI

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    if not settings.jwt_secret:
        logger.warning('JWT_SECRET non configurato - auth disabilitata.')

    await create_pool()           # pool MySQL asincrono
    await ensure_memory_table()    # crea ai_memoria se mancante
    await ensure_embeddings_column() # aggiunge col. embedding a cani
    await ensure_cani_created_at() # aggiunge col. creato_at a cani
    await ensure_notifications_table() # crea ai_notifiche
    await ensure_knowledge_table() # crea ai_knowledge_base
    await seed_knowledge_from_json( # popola KB da JSON (solo se vuota)
        Path(__file__).parent / 'knowledge_base.json')
    await ensure_booking_tables()  # crea aree_cani e prenotazioni
    await seed_aree_cani()         # inserisce 6 parchi (solo se vuoti)

    bg_task = asyncio.create_task(notification_loop()) # avvia scanner
    yield # app e' viva da qui

    bg_task.cancel()              # cancella il task
    try: await bg_task
    except asyncio.CancelledError: pass
    await close_pool()            # chiude MySQL pool
```

Nota: La chiamata ensure_embeddings_column() aggiunge le colonne 'embedding' (MEDIUMBLOB) e 'embedding_hash' (VARCHAR 64) alla tabella 'cani' del database condiviso con backend_node. Questo è l'unico punto in cui dogmatch_ai modifica lo schema di una tabella che non ha creato lui.

1.2 Autenticazione JWT opzionale

Ogni endpoint che richiede autenticazione chiama la funzione _verify_jwt(). Il meccanismo è volutamente opzionale: se JWT_SECRET non è configurato nel .env, la funzione ritorna immediatamente senza verificare nulla. Questo permette di usare il servizio in sviluppo senza configurare il JWT, ma in produzione il JWT_SECRET deve essere sempre impostato. Il token viene estratto dall'header Authorization con il prefisso 'Bearer'.

main.py - _verify_jwt

```
def _verify_jwt(authorization: Optional[str]) -> None:
    if not settings.jwt_secret:
        return # auth disabilitata: non verifica nulla

    if not authorization or not authorization.startswith('Bearer '):
```

```

    raise HTTPException(status_code=401,
                        detail='Token di autenticazione mancante.')

token = authorization.removeprefix('Bearer ').strip()
try:
    jwt.decode(token, settings.jwt_secret,
               algorithms=[settings.jwt_algorithm])
except jwt.ExpiredSignatureError:
    raise HTTPException(status_code=401, detail='Token scaduto.')
except jwt.InvalidTokenError:
    raise HTTPException(status_code=403, detail='Token non valido.')

```

1.3 Rate limiting per utente

Prima di passare la richiesta all'orchestratore, il servizio controlla il rate limit tramite `check_rate_limit()`. L'implementazione usa un `defaultdict` Python che mappa ogni `user_id` a una lista di timestamp. Ad ogni chiamata, i timestamp vecchi (piu' di 60 secondi) vengono rimossi, e se la lista supera 10 elementi viene sollevata una `HTTPException 429`.

rate_limit.py

```

_WINDOW = 60 # secondi
_MAX = 10 # richieste massime per finestra
_buckets: defaultdict[str, list] = defaultdict(list)

def check_rate_limit(user_id: str) -> None:
    now = datetime.now(UTC)
    cutoff = now - timedelta(seconds=_WINDOW)

    # Rimuove i timestamp fuori dalla finestra
    _buckets[user_id] = [t for t in _buckets[user_id] if t > cutoff]

    if len(_buckets[user_id]) >= _MAX:
        raise HTTPException(
            status_code=429,
            detail=f'Troppe richieste. Massimo {_MAX} al minuto.',
        )
    _buckets[user_id].append(now) # registra questa richiesta

```

Attenzione: I bucket sono in memoria del processo. Su piu' istanze del servizio ogni istanza avrebbe un contatore separato e il limite sarebbe moltiplicato per il numero di istanze. Per scalare orizzontalmente occorre un backend condiviso come Redis.

1.4 I modelli Pydantic

FastAPI usa Pydantic per validare automaticamente i dati in ingresso e in uscita. I modelli sono definiti in `models.py`. `ChatRequest` impone un limite di 500 caratteri sul messaggio tramite il parametro `max_length` di `Field`. Il campo `'context'` e' la stringa di system prompt pre-costruita dal backend Node.js, che include profilo utente e dati dal database.

models.py

```

class HistoryMessage(BaseModel):
    role: str # 'user' | 'assistant'
    content: str

class ChatRequest(BaseModel):
    message: str = Field(..., max_length=500) # max 500 caratteri
    user_id: str
    context: str # system prompt pre-costruito da backend_node
    history: List[HistoryMessage] = []

class ChatResponse(BaseModel):

```

```
risposta: str # risposta finale del bot
```

2. L'orchestratore: il cuore del servizio

orchestrator.py e' il modulo piu' complesso e importante del servizio. Implementa il ciclo completo: rilevamento dell'intenzione dell'utente, pre-fetch dei risultati via RAG, costruzione del system prompt e il loop di tool-calling con GPT-4o. Tutto e' progettato per essere asincrono (async/await) per non bloccare il server mentre aspetta risposte dal database o da OpenAI.

2.1 Intent detection: capire se l'utente cerca un cane

Prima di costruire il prompt, l'orchestratore deve capire se il messaggio e' una richiesta di ricerca cani. Questo e' importante perche' se lo e', conviene pre-caricare i risultati nel prompt invece di aspettare che GPT-4o chiami il tool search_dogs_semantic. La funzione _is_dog_search usa due strategie in base alla disponibilita' di sentence-transformers: embedding-based o keyword fallback.

orchestrator.py - calcolo centroide al boot

```
# 10 frasi prototipo che descrivono l'intenzione di cercare un cane
_INTENT_EXAMPLES = [
    'cerco un cane compatibile con il mio',
    'trova cani adatti per il mio cane',
    'suggerisci cani simili al mio',
    ... # 10 esempi totali
]
_INTENT_THRESHOLD = 0.35 # soglia cosine similarity

# Al caricamento del modulo: calcola il centroide degli esempi
if _USE_EMBEDDINGS:
    _vecs = _encode(_INTENT_EXAMPLES) # vettori 384-dim
    _intent_centroid = _vecs.mean(axis=0)
    _intent_centroid /= (np.linalg.norm(_intent_centroid) or 1e-8)
```

Il centroide degli esempi viene calcolato una sola volta al caricamento del modulo, non ad ogni richiesta. Ad ogni messaggio in arrivo, il suo embedding viene confrontato con questo centroide tramite cosine similarity: se il punteggio supera 0.35, e' una ricerca cani. Se sentence-transformers non e' disponibile, si usa un semplice controllo di stem (radici di parole) come fallback.

orchestrator.py - _is_dog_search

```
async def _is_dog_search(message: str) -> bool:
    if _USE_EMBEDDINGS and _intent_centroid is not None:
        # Calcola embedding in thread separato (operazione CPU-bound)
        emb = await asyncio.to_thread(_encode, [message.lower()])
        emb_norm = emb[0] / (np.linalg.norm(emb[0]) or 1e-8)
        score = float(np.dot(emb_norm, _intent_centroid))
        logger.debug('Intent score: %.3f', score)
        return score >= _INTENT_THRESHOLD # True se >= 0.35

    # Fallback keyword: controlla 29 stem italiani
    msg = message.lower()
    return any(stem in msg for stem in _SEARCH_STEMS)
    # es: 'cerc', 'trov', 'match', 'compagn', 'mite', 'energic'...
```

Nota: asyncio.to_thread() e' fondamentale: sentence-transformers e' una libreria CPU-bound (non rilascia il GIL durante l'encoding). Chiamarla direttamente in un contesto async bloccherebbe l'event loop di FastAPI impedendo di servire altre richieste nel frattempo.

2.2 RAG pre-fetch: caricare i risultati prima di GPT-4o

Se il messaggio e' una ricerca cani, _rag_context() recupera il profilo del cane dell'utente dal database e lo usa per arricchire la query. Per esempio, se l'utente ha un Labrador maschio di 3 anni e scrive 'trovami un compagno per giocare', la query

arricchita diventa 'trovami un compagno per giocare Labrador maschio 3 anni Media'. Questo re-ranking implicito porta risultati piu' rilevanti.

orchestrator.py - _rag_context

```
async def _rag_context(message: str, user_id: str) -> str:
    if not await _is_dog_search(message):
        return '' # non e' una ricerca: nessun pre-fetch

    profile = await _fetch_user_dog_profile(user_id)
    # Query arricchita con il profilo del cane dell'utente
    enriched = f'{message} {profile}'.strip() if profile else message

    result = await search_dogs_semantic(query=enriched, user_id=user_id)

    if result.startswith('Nessun cane') or result.startswith('Non ho trovato'):
        return ''

    return (
        '--- Cani pre-caricati in base alla richiesta ---\n'
        + result + '\n---\n'
        'Questi risultati sono già disponibili nel contesto. '
        'Usali per rispondere direttamente... '
        'senza chiamare search_dogs_semantic.'
    )
```

La stringa restituita da `_rag_context` viene inserita nel system prompt con un'istruzione esplicita a GPT-4o: usa questi risultati per rispondere senza chiamare di nuovo il tool di ricerca, a meno che l'utente non specifichi filtri particolari. Questo riduce il numero di tool-call e quindi il tempo di risposta.

2.3 Costruzione del system prompt

La funzione `_build_messages` assembla la lista di messaggi che viene inviata a GPT-4o. Il system prompt e' composto concatenando piu' sezioni in un ordine preciso, ognuna separata da due a capo. Le notifiche non lette dell'utente, se presenti, vengono inserite con un'istruzione a menzionarle all'inizio della risposta in modo naturale. Il RAG (cani pre-caricati) viene aggiunto alla fine del system, prima della history, in modo che GPT-4o lo veda come contesto recente.

orchestrator.py - _build_messages

```
def _build_messages(request, user_name, memory, rag, notifications):
    parts = [request.context, BOT_IDENTITY] # 1. context + identità

    if user_name:
        parts.append(f'Stai parlando con {user_name}.') # 2. nome utente

    if memory:
        parts.append(memory) # 3. memoria AI

    if notifications:
        notif_lines = '\n'.join(f'- {n}' for n in notifications)
        parts.append(
            f'--- {len(notifications)} nuova/e notifica/he ---\n'
            f'{notif_lines}\n---\n'
            'Menziona queste notifiche in modo naturale...'
        ) # 4. notifiche

    if rag:
        parts.append(rag) # 5. RAG

    # Il system prompt e' tutto il testo unito
    messages = [{'role': 'system', 'content': '\n\n'.join(parts)}]
```

```

for h in request.history:                                # 6. history
    messages.append({'role': h.role, 'content': h.content})

messages.append({'role': 'user', 'content': request.message}) # 7.
return messages

```

2.4 Il loop di tool-calling con GPT-4o

Il cuore dell'orchestratore è `get_ai_response()`, che esegue un loop fino a `MAX_TOOL_ITERATIONS` (default 5). Ad ogni iterazione chiama l'API di OpenAI e analizza il `finish_reason` della risposta. Se è `'stop'`, GPT-4o ha finito: si ritorna il testo. Se è `'tool_calls'`, GPT-4o ha deciso di chiamare uno o più tool: il sistema li esegue e aggiunge i risultati alla lista dei messaggi per la prossima iterazione. Se è `'length'`, la risposta è stata troncata per limiti di token.

orchestrator.py - get_ai_response

```

async def get_ai_response(request: ChatRequest) -> str:
    # Carica tutto in parallelo: non aspetta un risultato alla volta
    user_name, memory, rag, notifications = await asyncio.gather(
        _fetch_user_name(request.user_id),
        load_user_memory(request.user_id),
        _rag_context(request.message, request.user_id),
        load_unread_notifications(request.user_id),
    )
    messages = _build_messages(request, user_name, memory, rag, notifications)

    for iteration in range(settings.max_tool_iterations):
        response = await _client.chat.completions.create(
            model=settings.model,
            max_tokens=settings.max_tokens,
            messages=messages,
            tools=TOOLS,
            tool_choice='auto', # GPT-4o decide se/quando usare tool
        )
        choice = response.choices[0]

        if choice.finish_reason == 'stop':
            return choice.message.content # risposta finale

        if choice.finish_reason == 'tool_calls':
            messages.append(choice.message) # aggiunge msg del modello
            for tool_call in choice.message.tool_calls:
                tool_input = json.loads(tool_call.function.arguments)
                logger.info('Tool call: %(s)s',
                    tool_call.function.name, tool_input)
                result = await execute_tool(
                    tool_call.function.name, tool_input, request.user_id)
                messages.append({ # aggiunge risultato del tool
                    'role': 'tool',
                    'tool_call_id': tool_call.id,
                    'content': result
                })
            # continua il loop

        elif choice.finish_reason == 'length':
            return 'Risposta troncata. Riprova con una domanda piu specifica.'

    return 'Mi dispiace, non sono riuscito a elaborare la risposta.'

```

Nota: `asyncio.gather()` esegue le quattro operazioni di caricamento (nome utente, memoria, RAG, notifiche) in parallelo. Anziché aspettarle

in sequenza, FastAPI avvia tutte e quattro le query al database contemporaneamente e aspetta che terminino tutte. Questo riduce il tempo di attesa al massimo tra le quattro (invece della somma).

3. L'executor dei tool

Quando GPT-4o decide di chiamare un tool, l'orchestratore passa il nome e gli argomenti alla funzione `execute_tool()` dell'executor. L'executor usa un dizionario `_TOOL_MAP` che mappa ogni nome di tool a una lambda che chiama la funzione Python corrispondente. Questo pattern tiene tutto il routing dei tool in un unico posto, rendendo facile aggiungere o rimuovere tool.

tools/executor.py

```
_TOOL_MAP = {
    'search_dogs_semantic': lambda inp, uid: search_dogs_semantic(
        query=inp['query'],
        filters=inp.get('filters'),
        user_id=uid,
    ),
    'save_memory': lambda inp, uid: save_memory(
        user_id=uid,
        chiave=inp['chiave'],
        valore=inp['valore'],
    ),
    'get_knowledge_base_info': lambda inp, uid: get_knowledge_base_info(
        topic=inp['topic'],
    ),
    'moderation_flag': lambda inp, uid: moderation_flag(
        content=inp['content'],
        reason=inp['reason'],
        user_id=uid,
    ),
    # ...altri 10 tool...
}

async def execute_tool(name: str, tool_input: dict, user_id: str) -> str:
    handler = _TOOL_MAP.get(name)
    if handler is None:
        return f'Strumento {name!r} non disponibile.'
    try:
        return await handler(tool_input, user_id)
    except KeyError as e:
        return f'Parametro mancante per {name}: {e}.'
    except Exception as e:
        logger.error('Errore nel tool %s: %s', name, e)
        return f'Errore durante l'esecuzione di {name}. Riprova.'
```

La gestione degli errori nell'executor e' progettata per non far crashare il loop: se un tool fallisce, restituisce una stringa di errore. GPT-4o riceve questo messaggio come risultato del tool e puo' decidere di riprovare o di rispondere comunque all'utente spiegando il problema.

4. La memoria persistente del bot

SnoutBot ha una memoria persistente per utente, salvata nella tabella `ai_memoria` del database MySQL. La memoria serve per ricordare informazioni sull'utente tra sessioni diverse: la razza preferita, l'obiettivo (gioco vs accoppiamento), il nome del cane, ecc. Si distinguono due tipi di record: fatti normali e promemoria.

4.1 Fatti e promemoria: la stessa tabella, chiavi diverse

Entrambi i tipi usano la tabella `ai_memoria` con le colonne (`user_id`, `chiave`, `valore`). La distinzione avviene tramite il prefisso della chiave: i promemoria hanno chiave `'todo_<id>'`, i fatti no. Al caricamento della memoria, `load_user_memory()` separa le due categorie e le formatta in due blocchi di testo distinti, inseriti nel system prompt.

tools/memory.py - load_user_memory

```
async def load_user_memory(user_id: str) -> str:
    async with get_db() as cur:
        await cur.execute(
            'SELECT chiave, valore FROM ai_memoria '
            'WHERE user_id = %s ORDER BY aggiornato_at DESC',
            (user_id,),
        )
        rows = await cur.fetchall()
    if not rows:
        return '' # nessuna memoria: nessun testo aggiunto al prompt

    # Separa fatti (no prefisso todo_) da promemoria (prefisso todo_)
    facts = [(r['chiave'], r['valore'])
              for r in rows if not r['chiave'].startswith('todo_')]
    todos = [(r['chiave'].removeprefix('todo_'), r['valore'])
              for r in rows if r['chiave'].startswith('todo_')]

    parts = []
    if facts:
        lines = '\n'.join(f'- {k}: {v}' for k, v in facts)
        parts.append(f'Ricordi su questo utente:\n{lines}')
    if todos:
        lines = '\n'.join(f'- [{id}] {testo}' for id, testo in todos)
        parts.append(
            f'Promemoria in sospeso '
            f'(ricordali proattivamente):\n{lines}'
        )
    return '\n\n'.join(parts)
```

4.2 Salvataggio con upsert

Il salvataggio di un fatto o di un promemoria usa `INSERT ... ON DUPLICATE KEY UPDATE`. Questo significa che se la chiave esiste già per quell'utente, il valore viene aggiornato invece di creare un duplicato. La coppia (`user_id`, `chiave`) è l'indice unico della tabella.

tools/memory.py - save_memory, save_reminder, clear_reminder

```
async def save_memory(user_id: str, chiave: str, valore: str) -> str:
    async with get_db() as cur:
        await cur.execute(
            '''INSERT INTO ai_memoria (user_id, chiave, valore, aggiornato_at)
            VALUES (%s, %s, %s, NOW())
            ON DUPLICATE KEY UPDATE'''
```

```

        valore = VALUES(valore),
        aggiornato_at = NOW()'',
        (user_id, chiave, valore),
    )
    return f'Ricordato: {chiave} -> {valore}'

async def save_reminder(user_id: str, id: str, testo: str) -> str:
    # Usa chiave 'todo_<id>' per distinguere i promemoria
    async with get_db() as cur:
        await cur.execute(
            '...ON DUPLICATE KEY UPDATE...',
            (user_id, f'todo_{id}', testo), # prefisso todo_
        )
    return f'Promemoria salvato: ricordero di {testo}'

async def clear_reminder(user_id: str, id: str) -> str:
    async with get_db() as cur:
        await cur.execute(
            'DELETE FROM ai_memoria WHERE user_id = %s AND chiave = %s',
            (user_id, f'todo_{id}'),
        )
    return f'Promemoria {id!r} completato e rimosso.'
```

5. Le notifiche proattive: il background scanner

Una delle funzionalita' piu' interessanti di dogmatch_ai e' il sistema di notifiche proattive. Un loop asincrono scansiona ogni 5 minuti i nuovi cani registrati sulla piattaforma e, per ciascuno, controlla se e' compatibile con le preferenze degli utenti che hanno salvato qualcosa in memoria. Se la similarity supera 0.45, genera una notifica personalizzata che l'utente vedra' all'apertura successiva della chat.

5.1 Il loop asincrono

Il `notification_loop()` in `background.py` e' un ciclo `while True` che chiama `run_notification_scan()` e poi aspetta 300 secondi. E' un `asyncio.Task` avviato durante il lifespan e cancellato allo shutdown. Gli errori vengono loggati ma non interrompono il loop.

background.py

```
_SCAN_INTERVAL = 300 # 5 minuti

async def notification_loop() -> None:
    while True:
        try:
            await run_notification_scan()
        except Exception as exc:
            logger.error('Errore nel notification scanner: %s', exc)
        await asyncio.sleep(_SCAN_INTERVAL) # aspetta 5 minuti
```

5.2 La logica di scansione e matching

`run_notification_scan()` in `notifications.py` implementa la logica vera. Al primo avvio imposta solo il timestamp baseline senza fare nulla (non esistono cani 'nuovi' prima del riferimento). Nelle esecuzioni successive carica i cani registrati dopo l'ultimo scan e gli utenti con preferenze in memoria. Per ogni combinazione (nuovo_cane, utente), calcola la similarity tra il testo del cane e le preferenze dell'utente.

tools/notifications.py - run_notification_scan

```
_MATCH_THRESHOLD = 0.45
_last_scan: Optional[datetime] = None

async def run_notification_scan() -> None:
    global _last_scan
    now = datetime.now(UTC)

    if _last_scan is None:
        _last_scan = now # primo avvio: imposta baseline, non scansiona
        return

    new_dogs = await _new_dogs_since(_last_scan)
    if not new_dogs: return # nessun nuovo cane: skip

    user_ids = await _users_with_preferences()
    # SELECT DISTINCT user_id FROM ai_memoria WHERE chiave NOT LIKE 'todo_%'

    for dog in new_dogs:
        dog_text = _build_dog_text(dog) # testo descrittivo del cane

        # Batch encode: cane + tutte le preferenze utenti insieme
        all_texts = [dog_text] + [pref for _, pref in user_prefs]
        embeddings = await asyncio.to_thread(_encode, all_texts)
```

```

dog_emb = embeddings[0] / (np.linalg.norm(embeddings[0]) or 1e-8)

for (uid, _), pref_emb in zip(user_prefs, embeddings[1:]):
    pref_norm = pref_emb / (np.linalg.norm(pref_emb) or 1e-8)
    score = float(np.dot(dog_emb, pref_norm)) # cosine similarity

    if score >= _MATCH_THRESHOLD:
        msg = (
            f'Nuovo cane compatibile: **{dog['nome']}** '
            f'({dog['razza']}, {dog.get('eta','?')} anni) - '
            f'Affinita: {int(score * 100)}%'
        )
        await _save_notification(uid, dog['id'], msg)
_last_scan = now

```

Il batch encode e' fondamentale per le performance: invece di calcolare l'embedding del cane una volta per ogni utente (N chiamate al modello), tutti i testi (cane + preferenze di tutti gli utenti) vengono encodati in una sola chiamata. Il salvataggio usa INSERT IGNORE per evitare notifiche duplicate: se la stessa coppia (user, cane) ha gia' una notifica, non ne viene creata un'altra.

5.3 Lettura delle notifiche nella chat

Quando un utente apre la chat, `load_unread_notifications()` carica le notifiche non lette e le marca immediatamente come lette (`letta=1`) nella stessa transazione. Questo garantisce che ogni notifica venga mostrata una sola volta, anche se l'utente invia piu' messaggi nella stessa sessione.

tools/notifications.py - load_unread_notifications

```

async def load_unread_notifications(user_id: str) -> list[str]:
    async with get_db() as cur:
        await cur.execute(
            'SELECT id, messaggio FROM ai_notifiche '
            'WHERE user_id = %s AND letta = 0',
            (user_id,),
        )
        rows = await cur.fetchall()
        if rows:
            ids = [r['id'] for r in rows]
            placeholders = ','.join(['%s'] * len(ids))
            # Marca lette nello stesso contesto DB
            await cur.execute(
                f'UPDATE ai_notifiche SET letta = 1 '
                f'WHERE id IN ({placeholders})', ids
            )
        return [r['messaggio'] for r in rows]

```

6. Moderazione: audit log append-only

Il tool `moderation_flag` non blocca l'utente segnalato ne' intraprende azioni immediate. Si limita a scrivere una riga JSON nel file `moderation_log.jsonl` (append-only) e a restituire un messaggio di conferma all'utente. Il team di moderazione esamina il log e decide le azioni da intraprendere. Questo approccio e' deliberato: un blocco automatico potrebbe essere troppo aggressivo su falsi positivi.

tools/moderation.py

```
_LOG_PATH = Path(__file__).parent.parent / 'moderation_log.jsonl'

async def moderation_flag(content: str, reason: str, user_id: str) -> str:
    flag_id = str(uuid.uuid4())[:12].upper() # ID compatto leggibile
    timestamp = datetime.now(timezone.utc).isoformat()

    entry = {
        'flag_id':      flag_id,
        'timestamp':    timestamp,
        'user_id':      user_id,
        'reason':       reason,
        'content_preview': content[:200], # solo prime 200 chars
    }

    # Append-only: non sovrascrive mai il file
    with open(_LOG_PATH, 'a', encoding='utf-8') as f:
        f.write(json.dumps(entry, ensure_ascii=False) + '\n')

    logger.warning('[MODERATION] flag_id=%s user=%s reason=%s',
                   flag_id, user_id, reason)

    return (
        f'Contenuto segnalato (ID: {flag_id}).\n'
        'La segnalazione verra esaminata entro 24 ore.'
    )
```

7. Modalita' streaming: SSE token per token

L'endpoint POST /chat/stream usa Server-Sent Events (SSE) per inviare la risposta un token alla volta, man mano che GPT-4o la genera. `get_ai_response_stream()` e' un async generator che usa `stream=True` nell'API OpenAI. Il loop accumula i delta dei `tool_calls` in un dizionario indicizzato perche' arrivano in frammenti separati nella risposta streaming.

orchestrator.py - get_ai_response_stream

```
async def get_ai_response_stream(request) -> AsyncGenerator[str, None]:
    # Stesso setup del loop sincrono
    ...

    for iteration in range(settings.max_tool_iterations):
        stream = await _client.chat.completions.create(
            ..., stream=True
        )

        full_content = ''
        tool_calls_acc: dict = {} # accumula tool_calls frammentati
        finish_reason = None

        async for chunk in stream:
            choice = chunk.choices[0]
            delta = choice.delta

            if choice.finish_reason:
                finish_reason = choice.finish_reason

            if delta.content:
                full_content += delta.content
                # Emette ogni token come evento SSE
                yield f'data: {json.dumps({"token": delta.content})}\n\n'

            if delta.tool_calls:
                for tc in delta.tool_calls:
                    idx = tc.index # indice del tool call
                    if idx not in tool_calls_acc:
                        tool_calls_acc[idx] = {'id': '', 'name': '', 'arguments': ''}
                    # Accumula i frammenti
                    if tc.id: tool_calls_acc[idx]['id'] = tc.id
                    if tc.function.name:
                        tool_calls_acc[idx]['name'] += tc.function.name
                    if tc.function.arguments:
                        tool_calls_acc[idx]['arguments'] += tc.function.arguments

        if finish_reason == 'stop':
            yield 'data: [DONE]\n\n'
            return
        # ...gestione tool_calls e length analoga al loop sincrono
```

8. Flusso completo end-to-end

Per rendere tutto concreto, seguiamo passo per passo cosa succede quando un utente (Marco, con un Labrador maschio di 3 anni) scrive a SnoutBot: 'Trovami un cane energico con cui giocare'.

Flusso end-to-end: 'Trovami un cane energico'

```
FASE 1 - Ricezione richiesta
POST /chat (Authorization: Bearer eyJ...)
Body: { message: 'Trovami un cane energico con cui giocare',
        user_id: 'uuid-marco', context: '<prompt costruito da Node.js>',
        history: [] }

main.py:
_verify_jwt(authorization) -> JWT valido
check_rate_limit('uuid-marco') -> 3/10 richieste: ok
await get_ai_response(request)

FASE 2 - Preparazione parallela
asyncio.gather() lancia 4 coroutine contemporaneamente:
_fetch_user_name() -> 'Marco'
load_user_memory() -> 'Ricordi: razza_preferita: Labrador'
_rag_context():
_is_dog_search() -> embedding: score=0.71 >= 0.35 -> True
_fetch_user_dog_profile() -> 'Labrador maschio 3 anni Media'
search_dogs_semantic(query='Trovami un cane energico... Labrador maschio')
-> top-5 cani piu compatibili (embedding similarity)
ritorna blocco RAG con top-5 e istruzione 'non chiamare il tool'
load_unread_notifications() -> ['Nuovo cane: Rex Beagle, Affinita 78%']

FASE 3 - Costruzione system prompt (_build_messages)
system = [
    request.context,          # profilo Marco costruito da Node.js
    BOT_IDENTITY,             # 'Sei SnoutBot di DogMatch'
    'Stai parlando con Marco.', # nome utente
    'Ricordi: razza_preferita: Labrador', # memoria
    '--- 1 notifica ---\n- Nuovo cane Rex Beagle...', # notifica
    '--- Cani pre-caricati ---\n[top-5 cani]\n---\nUsa questi...', # RAG
]

FASE 4 - GPT-4o iterazione 1
tool_choice='auto'
finish_reason='stop' (i cani sono gia nel prompt, non serve tool)
response.content:
'Ciao Marco! Prima di rispondere: ho una notifica per te -
un nuovo Beagle energico e stato aggiunto vicino a te!
Per il tuo Labrador ti consiglio questi 3 compagni di gioco:
1. Bolt (Golden Retriever, 2 anni, Milano) - 89% affinita...
2. Luna (Beagle, 1 anno, Milano) - 82%...
3. Max (Husky, 3 anni, Sesto S.G.) - 76%...'

FASE 5 - Risposta
return ChatResponse(risposta='Ciao Marco! Prima...')
-> backend_node riceve {risposta: '...'}
-> MessaggioAI.bulkCreate([user_msg, assistant_msg])
-> io.emit('bot_typing', {isTyping: false})
-> client riceve il testo finale
```


9. Punti di attenzione

1. JWT_SECRET assente: autenticazione disabilitata

Se JWT_SECRET non e' nel .env, _verify_jwt() ritorna senza verificare nulla. Chiunque conosca l'IP e la porta 8000 puo' chiamare POST /chat. Sicuro solo se 8000 e' vincolata a 127.0.0.1 (non esposta su internet).

2. Rate limiting in-memory: non funziona su piu' istanze

_buckets e' un defaultdict Python in memoria del processo. Con due istanze del servizio (load balancer), ogni istanza ha il suo contatore separato e l'utente puo' fare 10+10=20 richieste. Per produzione scalabile: Redis con INCR/EXPIRE.

3. sentence-transformers ~471MB a cold start

Il modello paraphrase-multilingual-MiniLM-L12-v2 viene scaricato da HuggingFace Hub al primo avvio se non e' in cache. Su un server senza accesso a internet o con disco limitato questo e' un problema. Da includere nell'immagine Docker.

4. Embedding come MEDIUMBLOB in MySQL

I vettori 384-float32 serializzati occupano $384 \times 4 = 1536$ byte ciascuno. Su 100k cani sono circa 150MB solo di embedding nel database. Oltre questa soglia un vector store dedicato (pgvector, Weaviate) sarebbe piu' efficiente per le query di similarity.

5. MAX_TOOL_ITERATIONS fisso a 5

Richieste che richiedono molti tool (es. cerca cane + prenota area + salva nella memoria + aggiorna profilo) potrebbero non completarsi entro 5 iterazioni. Il risultato e' un messaggio di errore generico anziche' una risposta parziale utile.